AnalysisBank: An Expert Analysis Pattern Library for Financial Report Generation

Yajing Yang[1], **Yunshan Ma**[2], **Kelvin J.L. Koa**[1,3], **Min-Yen Kan**[1]

[1]National University of Singapore [2]Singapore Management University
[3]Asian Institute of Digital Finance

yajing.yang@u.nus.edu ; ysma@smu.edu.sg ; kelvin.koa@u.nus.edu ; kanmy@comp.nus.edu.sg

Abstract

We argue that financial report generation should operate at the analytical rather than structural level, composing content from data-derived insights rather than high-level topics or sections. To this end, we propose AnalysisBank, which distills expert reports into a reusable library of *Analyses*, each pairing a data signal, an analytical move, and the expert span it was derived from. At inference time, AnalysisBank matches input signals to library entries and applies the retrieved moves to compose the report. A study of *Analyses* distilled from 550 expert reports reveals a heavy-tailed distribution of 47–52 signal types spanning 13 move types. On two financial benchmarks across four LLM backbones, AnalysisBank increases the proportion of novel, data-grounded insights by 1.7–3.7× over structural-level baselines. Transfer to scientific writing suggests that the distinction generalizes beyond finance. Code and the distilled *Analysis* library are available at https://github.com/yajingyang/AnalysisBank.

1 Introduction

In finance, analytical reports support decisions by producing *insights*: claims derived from analyzing source data rather than paraphrasing it (Asquith et al., 2005; Huang et al., 2014). Each insight is the output of an *analytical move*: a comparison, an attribution, a derivation, a projection, a gap-detection. Which analytical moves to make depends on what the data contains: the same earnings transcript, read by different analysts, yields different insights because each analyst recognizes different conditions in the data and responds with different analytical moves (Huang et al., 2018). This distinguishes the task from summarization (Lewis et al., 2020) and data-to-text generation (Wiseman et al., 2017), where the output paraphrases or restates the input; analytical report generation produces content the input does not state.

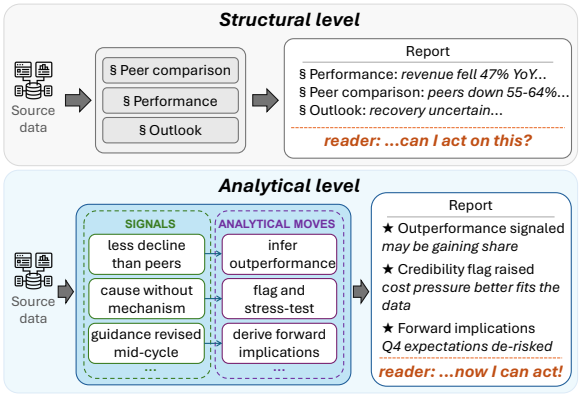


Figure 1: **Structural vs. analytical report generation.** Structural level (top): methods prescribe sections but leave the reasoning to model defaults. Analytical level (bottom): data signals are matched to the analytical moves they call for, yielding data-driven insights.

Existing methods operate at what we term the *structural level*: they prescribe what the report should cover, such as topics, viewpoints, or analytical categories, but leave the reasoning within each section to the language model's default behavior (Goldsack et al., 2025; Koshkin et al., 2025; Yang et al., 2022; Shao et al., 2024) (Figure 1, top). In every case the skeleton varies but the analytical substance does not: sections are filled with generic analyses such as trend description and standard ratio computation, regardless of what the input data specifically warrants. We argue that the task requires operating at the *analytical level*: the report should be composed of analytical moves driven by the specific signals in the source data (Figure 1, bottom). For example, a revenue decline smaller than peers' calls for a relative-outperformance inference; a downward guidance revision calls for a margin-implication derivation. These are not generic trend descriptions but signal-specific insights.

Operating at the analytical level is non-trivial for two reasons. First, the space of pairings between data signals and the analytical moves they call for is large and heavy-tailed: our corpus analysis iden-

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 22870–22892
October 24-29, 2026 ©2026 Association for Computational Linguistics

tifies 47–52 signal types spanning 13 analytical move types, yet over a third appear in three or fewer instances, each pairing with only one to two moves (§3.3). These tail patterns reflect the diverse ways experts respond to uncommon data conditions (Huang et al., 2018) and are too sparse for LLMs to learn from prompting or fine-tuning alone. Second, even given comprehensive pairings, selecting the right one for a given input is challenging: as autoregressive models, LLMs are biased toward high-probability outputs (McCoy et al., 2024) and default to generic analytical moves rather than the rare, signal-specific move the data calls for. Our ablation confirms that alternate selection mechanisms yield 19% less insightful content (§6.1).

To close this gap, we propose ANALYSISBANK. The core idea is that expert reports already embody analytical-level reasoning: each insight is derived from a specific data signal with the analytical move it calls for. ANALYSISBANK distills these pairings into a reusable library, where each entry, an *Analysis*, tuples a data signal, an analytical move, and the expert span it was distilled from, with the signal and move abstracted to making each *Analysis* reusable across inputs sharing the same pattern. Our study shows that *Analyses* distilled from 550 expert reports cover the heavy-tailed distribution (§3.3). The same abstraction addresses the selection challenge: by disentangling the underlying analytical pattern from surface specifics, the data signal field makes the link between a stored pattern and the input signal it should match explicit, enabling direct retrieval at inference time. On two benchmarks covering structured market data and unstructured earnings transcripts, ANALYSISBANK increases the proportion of novel, data-grounded insights by 1.7–3.7× across four LLM backbones.

Our contributions are threefold: (i) we distinguish *structural* and *analytical* levels of report generation and argue existing methods address only the former; (ii) we propose ANALYSISBANK, a library of expert-distilled *Analyses* and a pipeline to apply them at inference; (iii) on two benchmarks across four backbones, ANALYSISBANK increases novel, data-grounded insight by 1.7–3.7×, while structural-level methods converge well below it.

2 Related Work

We break down relevant prior work into three areas: long-form and report generation (the task), reusable reasoning patterns (the method), and financial NLP (the domain).

Long-form and report generation. Data-to-text generation maps tables and records to fluent prose (Wiseman et al., 2017; Parikh et al., 2020), but the output restates the input rather than analyzing it. Report-generation methods go further by introducing organizational structure: multi-agent frameworks assign predefined analytical roles (Goldsack et al., 2025; Koshkin et al., 2025), plan-then-write approaches decompose output into sections (Yang et al., 2022; Puduppully et al., 2019; Hu et al., 2022), and retrieve-organize-write systems research a topic through retrieval and generate from the resulting outline (Shao et al., 2024). All three control what the report covers but not what analysis to apply to the data.

Reusable reasoning patterns. A natural approach is to retrieve reusable reasoning patterns at inference time. Self-discovered reasoning modules (Zhou et al., 2024) and thought templates (Yang et al., 2024a; Jeong et al., 2025) provide abstract reasoning strategies not grounded in specific data patterns. Agent workflows (Wang et al., 2025, 2024a) and induced functions (Wang et al., 2024b) learn concrete routines from task-solving trajectories but capture full procedures, not individual analytical moves. Case-based methods (Yasunaga et al., 2024; Wiratunga et al., 2024) and experience memories (Park et al., 2023; Zhong et al., 2024) operate at the instance level but store complete cases tied to specific inputs, not abstracted into reusable patterns. Analytical report generation requires patterns that are grounded in data, decomposed at the analytical-move level, and abstracted for reuse across the heavy-tailed distribution found in expert reports.

Financial NLP. Domain-pretrained models (Wu et al., 2023; Yang et al., 2023; Xie et al., 2023) adapt language models to financial text but do not explicitly model signal-to-move reasoning. Signal-mining pipelines extract salient content within it (Ju et al., 2023; Lu et al., 2025) but do not prescribe what analysis a signal calls for. Downstream reasoning work targets question answering (Chen et al., 2021; Zhu et al., 2021; Islam et al., 2023; Choi et al., 2025), injecting expert structure through hand-authored workflows (Nitarach et al., 2025) or historical scaffolds (Singhal, 2025; Le, 2024), but produces closed-form answers, not open-ended reports. For report generation,

data_signal: A key revenue or fee line shows a large decline versus the prior period, but the decline is smaller than what was broadly expected and also smaller than the decline reported by a comparable peer facing similar conditions.
analytical_move: When a major revenue or fee category declines sharply, compare the actual change to pre-report expectations and to the change reported by close peers; if the decline is less severe than both, explicitly assess and quantify whether this indicates relative outperformance and potential share gains in that activity.
reference_text: "Even investment banking, widely expected to be a sore thumb for all banks this quarter, may have contributed to this Friday's bullish reaction. To be clear, revenues of $1.7 billion were small as fees sank by a whopping 47% YOY. But keep the following in mind: (1) expectations were highly de-risked ahead of the print, with Seeking Alpha contributor Cavenagh Research bracing for a fee drop greater than 50%, and (2) JPMorgan, the top Wall Street player in investment banking, may have stolen some market share away from its competitors once again. Citi (C), for example, reported Q3 investment banking revenues that were an astonishing 64% lower YOY."

Table 1: Example *Analysis*. The data signal with three co-occurring conditions (actual vs. prior period, actual vs. expectations, actual vs. peer) feeds one analytical move (inference of outperformance and share gains).

bullet-point earnings-call summarization (Mukherjee et al., 2022) remains structural, while hierarchical data narration (Yang et al., 2025) reaches the analytical level but leaves the choice of analysis to the model, which defaults to the most common ones.

3 AnalysisBank

We first define the representation of each *Analysis*, designed to be reusable across inputs while preserving actionability (§3.1), then the extraction pipeline that populates the library from a corpus of expert reports (§3.2), and finally characterize the resulting library (§3.3).

3.1 *Analysis* Representation

Each entry in the AnalysisBank library, an *Analysis*, is a tuple of (`data_signal`, `analytical_move`, `reference_text`), as illustrated in Table 1.

The `reference_text` is a verbatim span from the source expert report that demonstrates the analytical move applied to real data (e.g., "revenues of $1.7 billion were small as fees sank by a whopping 47% YOY"). It provides a generation anchor and a faithfulness check, but contains specific entities, figures, and time references that prevent it from transferring to new inputs.

The `analytical_move` abstracts the reasoning in the reference text into a reusable instruction specifying what analysis to perform on the matched signal (e.g., "compare the actual change to expectations and to peers; if less severe than both, assess whether this indicates relative outperformance and potential share gains"). This transfers across inputs, but its specificity hinders fuzzy retrieval when used as the matching key.

The `data_signal` resolves this by serving as a separate retrieval key, stating triggering conditions in entity-free, number-free language (e.g., "a key revenue line declines sharply, but less than broadly expected and less than a comparable peer"). Retrieval matches on structural patterns while generation follows the specific instruction in the analytical move.

This three-field decomposition is a design hypothesis; we evaluate it against alternative designs in §6.1, finding that removing or merging any field degrades either retrieval precision or generation quality.

3.2 Extraction Pipeline

Prior pattern libraries induce entries from problem-solving traces with observable success signals (cf §2); expert reports offer no such structure, requiring the pipeline to reverse-engineer the three fields from finished prose. We design a four-pass pipeline to convert the corpus into a static library (Figure 2).

Induce identifies candidate *Analysis* content in each report. Analytical moves are implicit in expert prose; this stage makes them explicit by locating text spans where the analyst performed a specific reasoning operation. These spans become the `reference_text` of each *Analysis*.

Generalize extracts the `data_signal` and `analytical_move` from each identified span, producing the full three-field *Analysis* of §3.1. The data signal is abstracted by stripping entity names, specific figures, and time references while preserving the conditional structure. The analytical move is similarly abstracted into a specific instruction that applies to any instance of the signal. A self-check enforces that each *Analysis* would apply to the same pattern across different industries.

Deduplicate drops redundant *Analyses* across reports, since multiple expert reports often apply the same analytical move to similar signals. Candidates with `data_signal` embeddings above a co-

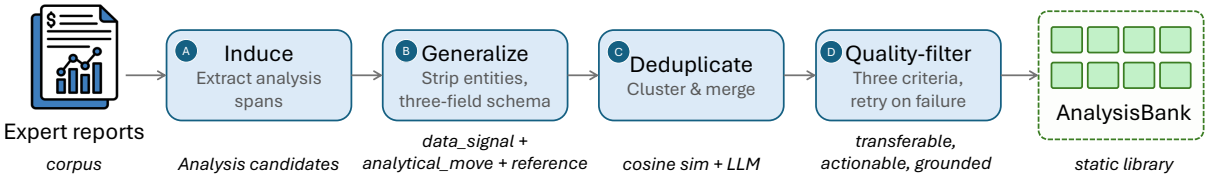


Figure 2: The four-pass ANALYSISBANK extraction pipeline, converting a corpus of expert reports into a static library of *Analyses*.

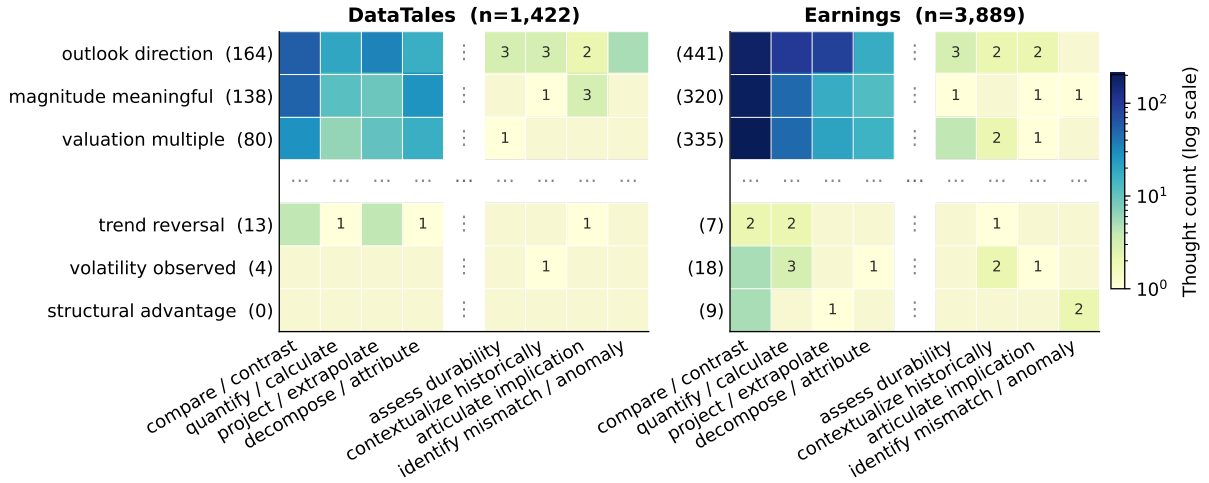


Figure 3: Signal-by-move distribution for three head and three tail signal types. Rows are signal types (parenthesized counts = total *Analyses*); columns are analytical-move types; cell color encodes *Analysis* count on a log scale. Full distribution in Appendix B.

sine similarity threshold are grouped together and merged into a single *Analysis*, retaining the most general data signal and the most complete analytical move. This prevents duplicate entries from consuming retrieval budget and biasing toward head patterns.

Quality-filter validates each *Analysis* against three criteria aligned with the three fields: transferability (does the data signal generalize across industries?), actionability (would two analysts following the analytical move independently produce similar outputs?), and grounding (does the reference text show a real instance of the pattern?). Grounding failures are discarded; transferability or actionability failures are routed back to Generalize with a targeted rewrite instruction. The resulting *Analyses* are industry-agnostic, executable, and empirically grounded.

The pipeline runs offline once, allowing a stronger model than the inference backbone to maximize *Analysis* quality. The resulting library is reused across all generation runs.

3.3 Coverage and Distribution

We instantiate ANALYSISBANK on two corpora spanning the dominant input modalities: 550 daily market-analysis reports written from structured price data (*DataTales*; Yang et al., 2024b) and 550 equity-research reports written from filings and earnings-call transcripts (*Earnings*[1]). The extraction pipeline of §3.2 yields 1,422 *Analyses* from DataTales and 3,889 from Earnings. We characterize each library by labeling every *Analysis* with a signal type and a move type using two hand-built keyword taxonomies (Appendix B). The move taxonomy has 13 types (e.g., compare/contrast), all populated in both corpora. The signal taxonomy has 55 types (e.g., outlook direction), of which 47 signal types are populated in DataTales and 52 in Earnings. The two axes are not independent: each signal type triggers a median of 4 distinct move types in DataTales and 6 in Earnings, and the move mix varies across signals (e.g., outlook direction spreads across compare/contrast, assess durability, contextualize historically, and articulate implication, while volatility observed concentrates on articulate implication alone; Figure 3).

This breadth coexists with a heavy long tail. The top-5 signals account for only 55% and 51% of

[1]The Earnings *Analysis* library is distilled from Seeking Alpha (https://seekingalpha.com/) analyst reports, which we cannot redistribute.

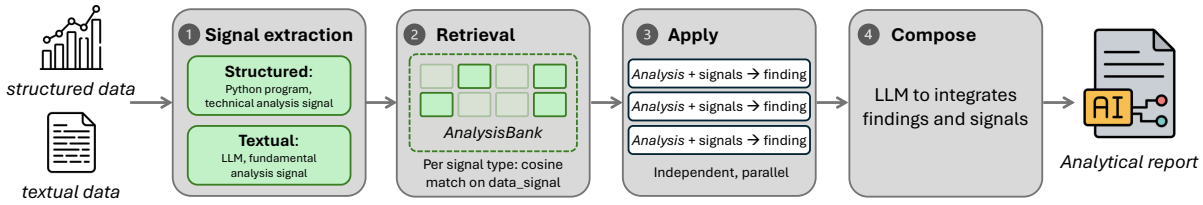


Figure 4: **Report generation pipeline.** A source input is reduced to typed signals (Stage 1), each signal type retrieves the most relevant *Analysis* from ANALYSISBANK library as constructed in Figure 2 (Stage 2), each retrieved *Analysis* is independently applied to produce a finding (Stage 3), and the findings are composed into a report (Stage 4).

Analyses in DataTales and Earnings respectively; reaching 90% coverage requires 19 and 22 signal types. The remaining tail is not residual noise: 17 of 47 signal types in DataTales (36%) and 10 of 52 in Earnings (19%) appear in $\leq$3 *Analyses*, yet unlike head signals that spread across many moves, each tail signal concentrates on 1–2 moves (Figure 3, below the dotted line), reflecting the narrow, decisive way an expert responds to an uncommon situation such as a trend reversal or a structural advantage assessment. These are precisely the patterns too rare for prompting or fine-tuning to internalize, preserved in reusable form by the abstraction of §3.1.

4 Narrative Generation with ANALYSISBANK

The library built in §3 provides coverage and the field structure needed for effective retrieval; the generation pipeline selects and applies the right *Analyses* for a given input (Figure 4).

Stage 1: Signal extraction. Raw financial input is too broad to match effectively against the abstracted `data_signal` field. This stage reduces it to a list of typed *signals*, each consisting of a signal type (e.g., margin delta, volatility regime), a signal description summarizing the condition, and supporting source spans. For textual input (e.g., earnings-call transcripts), an LLM extracts the signals, each verified by a fact-checking pass (Penman, 2013). For structured input (e.g., market data), a Python program computes them deterministically from price and volume series (Murphy, 1999) and formats the results into the signal description for retrieval.

Stage 2: Per-type retrieval. Signal descriptions query ANALYSISBANK by cosine similarity against `data_signal` field of each library entry. To ensure the retrieved *Analyses* cover diverse signal types rather than concentrating on one, each signal type contributes its best matching *Analysis*, and remaining slots are filled by global top-k.

Stage 3: Per-*Analysis* execution. Each retrieved *Analysis* is applied independently: an LLM call executes the `analytical_move` on the triggering signal descriptions and their supporting spans, producing one analytical finding. The `reference_text` is included as a demonstration of how the same type of analytical move has been applied before. Independence across calls ensures that Stage 4 receives findings with distinct analytical content.

Stage 4: Composition. A single LLM call composes the findings into a report. Rather than following a fixed section template, the model organizes around investment-relevant themes inferred from the findings themselves, consistent with the analytical-level principle that report structure should be driven by the data. The extracted signals from Stage 1 are included to fill factual gaps the findings do not cover, grounding the report in the source data.

The pipeline uses the same backbone model at every stage. Prompts and hyperparameters are in Appendix C. The contribution of each design choice across all stages is isolated in §6.1.

5 Experiments

Benchmarks. We evaluate on the two analytical report generation benchmarks. **DataTales** (Yang et al., 2024b) pairs structured market data with expert daily reports (460 instances). **Earnings2Insights** (Takayanagi et al., 2025) provides earnings call transcripts for analytical report generation (132 instances, extended from the original 64).

Models. We test four backbones spanning scale and capability: **Qwen3-8B** and **Qwen3.5-9B** (small open-source), **DeepSeek-V4-Flash** (reason-

Model	Method	DataTales					Earnings2Insights				
		%ins	%ana	depth	%fact	%win	%ins	%ana	depth	%fact	%win
Qwen3-8B	Direct	3.8	36.1	1.92	88.7	25.3	6.6	38.8	2.04	94.9	88.6
	CoT	7.2	48.8	2.16	**92.8**	26.5	4.2	38.3	2.15	94.5	58.8
	RAG	3.9	36.5	1.92	89.5	29.9	4.4	34.0	1.95	**96.1**	87.8
	BoT	9.2	48.2	2.18	88.9	40.9	5.2	31.3	1.55	91.8	86.3
	AWM	6.5	40.8	1.98	92.5	29.5	5.0	39.9	1.99	93.1	80.2
	AnalysisBank	**22.5**	**71.2**	**2.59**	92.1	**60.0**	**18.0**	**57.4**	**2.52**	89.8	**96.6**
Qwen3.5-9B	Direct	5.2	31.1	1.79	93.6	54.9	3.7	27.8	1.82	98.5	95.0
	CoT	5.7	35.5	1.84	95.2	51.6	6.4	34.0	1.93	97.6	88.2
	RAG	8.5	44.7	2.04	94.2	70.2	4.5	25.4	1.49	98.4	98.1
	BoT	14.0	50.4	2.19	95.8	**88.2**	3.2	19.1	1.66	98.8	74.6
	AWM	11.4	48.5	2.19	**96.7**	70.0	6.4	37.7	2.05	**98.9**	97.3
	AnalysisBank	**26.1**	**67.1**	**2.50**	89.8	81.9	**23.8**	**63.1**	**2.61**	97.2	**99.6**
DeepSeek -V4-Flash	Direct	5.5	36.7	1.79	94.0	75.6	5.7	34.6	1.92	**98.9**	97.7
	CoT	6.6	43.7	1.98	95.0	83.1	5.8	36.7	2.09	98.2	91.2
	RAG	7.9	46.9	2.10	92.4	85.4	7.5	39.5	2.12	98.5	98.1
	BoT	12.7	55.6	2.27	90.8	88.3	7.0	38.5	2.07	98.6	95.4
	AWM	10.5	52.4	2.25	**96.1**	96.3	7.7	42.7	2.07	97.0	98.5
	AnalysisBank	**21.1**	**67.6**	**2.48**	90.9	**99.3**	**18.1**	**55.5**	**2.36**	97.7	**100.0**
GPT-5.1	Direct	9.8	54.6	2.17	94.8	71.1	15.7	45.6	2.22	99.3	**100.0**
	CoT	4.0	28.8	1.57	94.8	43.3	14.2	42.2	2.30	**99.3**	**100.0**
	RAG	10.5	60.1	2.34	**95.9**	90.1	7.3	38.4	2.04	99.0	99.2
	AnalysisBank	**22.3**	**64.9**	**2.43**	95.5	**99.6**	**16.9**	**53.5**	**2.34**	98.5	99.2

Table 2: **Main results** across two benchmarks and four backbones. Columns: insight rate (%ins), analysis rate (%ana), reasoning depth (depth), factual precision (%fact), and win rate (%win). Insight rate is the headline metric; analysis rate is the control. Best per model in **bold**.

ing model), and **GPT-5.1** (proprietary), each in identical configurations across all conditions.

Baselines. We compare against two families. The prompting family consists **Direct**, **CoT**, and **RAG**. The structural-level pattern family consists of Buffer of Thoughts (**BoT**) (Yang et al., 2024a) and Agent Workflow Memory (**AWM**) (Wang et al., 2025), which are close to AnalysisBank in approach: BoT retrieves thought templates and AWM retrieves task-level workflows, both from a library at inference time. Both are adapted to distill from the same 550-report corpus that AnalysisBank consumes (adaptation details in Appendix D).

Metrics. We evaluate along five dimensions measuring the quality of analytical-level report generation. **Insight rate**, the fraction of claims that surface insights beyond what careful reading of the source data would yield (e.g., connecting a revenue decline to a peer's steeper decline to infer market share gain), is the headline metric. **Analysis rate**, the fraction of claims containing insights or standard expert analysis (e.g., decomposing that same revenue decline into volume and price components), measures the overall expert-level analytical content in the report. **Reasoning depth**, the mean analytical hops per claim, measures whether the gain comes from deeper inference chains. **Factual precision**, the fraction of numerical values correct against the source, ensuring analytical depth does not trade off correctness. **Win rate**, the rate at which the generated report is preferred over or tied with the expert reference, judged by two LLM personas (analyst, investor) under order-swapped trials. Judgments use Gemini-3-Flash; protocol details and reliability checks are in Appendix E, where the LLM judge's claim labels agree with human annotators at a level comparable to inter-annotator agreement, and human and judge insight rates correlate at the report level.

6 Results and Discussions

Main Results. Table 2 reports results across all conditions. AnalysisBank lifts insight rate by 1.7–3.7× over all baselines. AnalysisBank consistently outperforms all baselines:

On DataTales, insight rate (first column) rises from a best-baseline range of 9.2–14.0% to 21.1–26.1%; on Earnings2Insights, from 6.4–7.7% to 18.0–23.8% for the three non-GPT backbones. Baselines cluster in a narrow band regardless of prompting strategy: on DataTales with Qwen3-8B, insight rate rises by 2.4–5.9×, analysis rate by 1.3–2.1×, and reasoning depth by 0.4–0.7 hops, while

	Configuration	%insight	%analysis	depth	%factual	%win
Axis	Full pipeline (default)	23.8	63.1	2.61	97.2	99.6
Representation	Single-field	20.3 (+3.5)	58.1 (+5.0)	2.50 (+0.11)	98.2 (-1.0)	99.2 (+0.4)
	Two-field	23.9 (-0.1)	62.1 (+1.0)	2.57 (+0.04)	96.6 (+0.6)	97.6 (+2.0)
Retrieval target	Reference text	19.3 (+4.5)	56.8 (+6.3)	2.48 (+0.13)	96.4 (+0.8)	100.0 (-0.4)
Integration	Skip per-Analysis gen.	19.2 (+4.6)	57.9 (+5.2)	2.52 (+0.09)	98.1 (-0.9)	99.6 (0.0)
Composition	Transcript only	13.0 (+10.8)	49.1 (+14.0)	2.39 (+0.22)	98.6 (-1.4)	97.7 (+1.9)
	Signals only	11.1 (+12.7)	45.4 (+17.7)	2.21 (+0.40)	98.8 (-1.6)	96.6 (+3.0)
	Findings only	25.1 (-1.3)	65.9 (-2.8)	2.70 (-0.09)	94.1 (+3.1)	96.5 (+3.1)

Table 3: Ablation study on Earnings2Insights with Qwen3.5-9B. Each row changes one axis from the full pipeline. Green deltas indicate where the full pipeline is better; red deltas indicate where the variant is better.

factual precision remains comparable (89–97% across conditions) and win rate reaches 60–100%. Baselines cluster in a narrow band regardless of method: across all backbones, prompting methods fall within 4–16% insight rate of each other, and the structural-level methods (BoT, AWM) do not escape this band despite distilling from the same 550-report corpus, indicating a ceiling on insight for methods operating at the structural level.

To verify that insight gains stem from the library mechanism rather than prompting effects, we analyze 30 instances with the largest insight-rate gap between ANALYSISBANK and direct prompting. First, 80.4% of ANALYSISBANK's novel claims trace to a retrieved *Analysis* (44% strongly, 37% partially), confirming the library as the primary driver of novel content. Second, for 58% of these claims, the baseline either omits the same signal entirely (25%) or mentions it only generically (34%). The remaining 42% cite the same data but do not draw the analytical inference ANALYSISBANK makes, illustrating the selection challenge: the data is available but the right analytical move is not applied. Table 4 shows a representative traced claim alongside the baseline's treatment of the same input. Third, ANALYSISBANK covers 4.6 distinct signal types per instance vs. 0.83 for the baseline, a 5.5× gap. ANALYSISBANK produces novel insights across a wider range of data signals rather than concentrating on the most prominent one.

The gain is smallest for GPT-5.1 on Earnings2Insights, where Direct and CoT already reach 14–16% insight rate natively, leaving less headroom for the library. On DataTales, where no analysis is stated in the raw price and volume input, GPT-5.1 still gains 2.1× (22.3% vs. 10.5%), matching the lift on weaker backbones.

Retrieved *Analysis*	
signal	A company reports a very large amount of cash generated after necessary investments, and this amount is growing rapidly, while another well-known company with a much larger overall valuation generates only moderately more of this same type of cash flow.
move	Compare the company's cash generation level and growth to that of a significantly larger peer by relating each company's cash flow to its overall valuation, and use this comparison to assess how efficiently the company converts its value into cash relative to the peer.
Report generated for `PARA_q1_2022`	
Ours	"Operating cash flow grew 34% YoY to $347M, but it represents only 31.5% of the peer's $1.1B, suggesting lower cash flow efficiency."
Direct	Does not cite operating cash flow figures or any peer comparison.

Table 4: A novel claim traced to the *Analysis* that produced it (Earnings2Insights, `PARA_q1_2022`).

Human Evaluation. We perform two human evaluations to check that LLM-based metrics reflect genuine quality. To validate insight quality, two human annotators independently ranked 30 report triplets (ANALYSISBANK, best prompting baseline, BoT) by insight quality, blind to method identity. ANALYSISBANK is preferred in 96.7–100% of pairwise comparisons on Earnings2Insights and 63.3–86.7% on DataTales, confirming that ANALYSISBANK produces more insightful reports across both benchmarks. Annotators agree on which report ranks first in 66.7% of sets, with most disagreements concerning the ordering of the two baselines, which independently confirm the structural-level ceiling observed in the automated metrics.

To validate factuality, a human audit of sampled claims checks the content beyond the numerical values covered by %factual: ANALYSISBANK

contradicts the source in 4.7% of claims on the stronger backbone, below the pooled baseline, with the higher rate on Qwen3-8B tracking the reasoning capability that valid inference requires. Protocols and full results for both evaluations are in Appendix F.

6.1 Ablation Study

Table 3 isolates the contribution of each design choice by changing one axis at a time from the full pipeline (Qwen3.5-9B, Earnings2Insights).

Representation. Reducing to a single field (raw reference text only) drops insight rate by 3.5 points. Two analyses reveal why. First, at a cosine threshold of 0.85, the abstracted `data_signal` index retrieves 20× more library entries than `reference_text`, whose concrete prose produces a sparse embedding space with few retrievable neighbors. Second, 90% of entries retrieved via `data_signal` come from a different sector than the input, compared to 44% for `reference_text`, which clusters within the source domain. Abstraction makes the library globally reusable rather than locally bound. The two-field variant (fusing `data_signal` and `analytical_move` into one field) recovers most of the insight rate with a small drop on win rate (2 points), suggesting that separating the retrieval key from the generation instruction may further improves output quality.

Retrieval target. Retrieving on `reference_text` instead of `data_signal` drops insight rate by 4.5 points. Because `reference_texts` are concrete prose, similarity is driven by surface wording rather than by whether the stored pattern fits the input. For example, the input "E&I delivered a 6% volume increase" retrieves an entry whose reference span cites "a growth rate of 6% annually". The two share wording but not pattern: the analytical move retrieved converts forward expectations into an implied growth rate, while the input reports an actual change, so the move is not executable. Across reports, only 36.4% of the moves retrieved on `reference_text` are executed, against 52.8% for `data_signal`, which selects the more applicable move on 80% of reports.

Integration strategy. Skipping Stage 3 (per-*Analysis* generation) and passing retrieved *Analyses* directly to composition drops insight rate by 4.6 points. The `analysis_move` is an imperative that must be executed against the input's signals, not merely supplied as context; without this execution step, the composer defaults to the generic analytical moves that characterize the baseline ceiling.

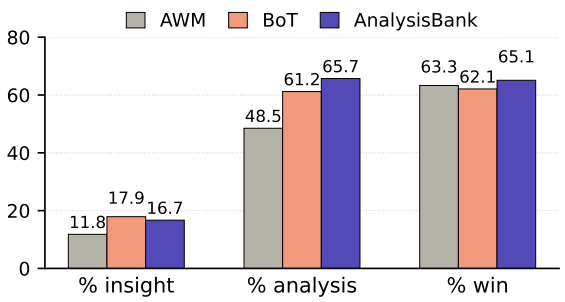


Figure 5: Cross-domain results on SciGen (Qwen3-8B). All three methods distill patterns from the same training corpus. Factual precision is comparable across methods (∼94%).

Composition input. Removing findings (the Stage 3 outputs produced by executing each retrieved analytical move) from Stage 4 halves insight rate to 11–13%, while the findings-only condition preserves it at 25.1%. This confirms that executed analytical moves, not raw input or the data signal extracted, derive insight content. The full pipeline adds signals and transcript for factual grounding: precision recovers from 94.1% to 97.2% and win rate from 96.5% to 99.6%, a deliberate trade-off for a modest 1.3-point insight cost.

6.2 Cross-Domain Transferability

The extraction pipeline and *Analysis* representation are designed to be domain-agnostic: the move typology, abstraction procedure, and quality filter make no assumptions specific to finance. To test this, we apply ANALYSISBANK to **SciGen** (Moosavi et al., 2021), a scientific paper generation benchmark, extracting *Analyses* from its training set and evaluating with Qwen3-8B against BoT and AWM on the same corpus (Figure 5).

The pattern partially replicates. ANALYSISBANK achieves the highest analysis rate (65.7% vs. 61.2% for BoT and 48.5% for AWM) and win rate (65.1%), confirming that the three-field decomposition transfers without architectural changes. On insight rate, BoT slightly edges ANALYSISBANK (17.9% vs. 16.7%), a reversal from the financial benchmarks where ANALYSISBANK leads by 1.7–3.7×. We attribute this to the nature of scientific writing: the dominant analytical moves (methodology comparison, result interpretation, limitation identification) are fewer and more predictable than

in financial analysis, reducing the long-tail advantage that drives ANALYSISBANK's gains on DataTales and Earnings2Insights. BoT's full-report templates are better matched to this more uniform move distribution, while ANALYSISBANK's per-*Analysis* decomposition adds coverage in the tail that SciGen's narrower move vocabulary does not reward as strongly. The key result is nonetheless positive: the extraction pipeline produces a functional library from a non-financial corpus, and the resulting reports achieve the highest overall quality without domain-specific adaptation.

7 Conclusion

Analytical report generation requires more than deciding what a report should cover: it requires determining what analyses the data warrants. Existing methods largely leave this reasoning to model defaults, leading to generic analytical behavior despite differences in report structure. ANALYSISBANK addresses this gap by retrieving and applying distilled expert *Analyses* at inference time, substantially increasing the proportion of novel, data-grounded insights across benchmarks and model backbones. Its transfer to scientific writing further suggests that operating at the analytical level may generalize beyond finance to other forms of expert, data-grounded reasoning.

Two directions extend this work. First, the current pipeline applies each *Analysis* independently in a single pass, but expert analysis is both iterative and compositional: one finding may trigger further Analyses, and some insights require chaining multiple moves into a compound finding. Iterative retrieval-generation (Shao et al., 2023; Trivedi et al., 2023) and analysis composition (Press et al., 2023) offer natural mechanisms but remain unexplored for analytical generation. Second, expert reports pair analysis with visualizations, yet the current pipeline produces text only. Extending the *Analysis* representation with visualization specifications (Yang et al., 2026) would enable multi-modal reports.

Limitations

The quality of the *Analysis* library is bounded by the expert reports it is distilled from. If the source reports are formulaic or lack deep analytical reasoning, the resulting *Analyses* will reflect those limitations. While our quality filter mitigates some of this risk, the pipeline cannot produce analytical patterns richer than what the source corpus contains. Our experiments are conducted exclusively on English corpora. The extraction pipeline relies on LLM-based abstraction and generalization, and its effectiveness on other languages has not been tested. ANALYSISBANK is designed for reports requiring data-specific analytical reasoning. For domains where a standard, fixed set of analyses suffices, the analytical-level approach offers limited advantage over structural-level methods, as the long-tail coverage that drives ANALYSISBANK's gains would not be needed.

Ethical Considerations

The expert reports used for *Analysis* extraction are publicly available financial documents. We release only the distilled *Analyses* and extraction code, not the source reports themselves, to respect the intellectual property of the original authors and publishers.

Our automated evaluation relies on LLM-based judgment. Human evaluation involved two voluntary annotators from the authors' professional network who ranked system-generated reports without compensation; no personal data was collected.

Generated reports may contain analytical errors or unsupported inferences. We strongly recommend human review before any investment decision is based on system-generated content, and advocate for human-AI collaboration where the system augments rather than replaces expert judgment. Any deployment of this technology should clearly disclose that reports are AI-generated to ensure transparency for readers and stakeholders.

References

Paul Asquith, Michael B. Mikhail, and Andrea S. Au. 2005. Information content of equity analyst reports. *Journal of Financial Economics*, 75(2):245–282.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Kenneth Huang, Bryan R. Routledge, and William Yang Wang. 2021. Finqa: A dataset of numerical reasoning over financial data. In *EMNLP (1)*, pages 3697–3711. Association for Computational Linguistics.

Chanyeol Choi, Jihoon Kwon, Jaeseon Ha, Hojun Choi, Chaewoon Kim, Yongjae Lee, Jy-yong Sohn, and Alejandro Lopez-Lira. 2025. Finder: Financial dataset for question answering and evaluating retrieval-augmented generation. In *ICAIF*, pages 638–646. ACM.

Alvan R Feinstein and Domenic V Cicchetti. 1990. High agreement but low kappa: I. the problems of two paradoxes. *Journal of clinical epidemiology*, 43(6):543–549.

Tomas Goldsack, Yang Wang, Chenghua Lin, and Chung-Chi Chen. 2025. From facts to insights: A study on the generation and evaluation of analytical reports for deciphering earnings calls. In *COLING*, pages 10576–10593. Association for Computational Linguistics.

Zhe Hu, Hou Pong Chan, Jiachen Liu, Xinyan Xiao, Hua Wu, and Lifu Huang. 2022. PLANET: dynamic content planning in autoregressive transformers for long-form text generation. In *ACL (1)*, pages 2288–2305. Association for Computational Linguistics.

Allen H. Huang, Reuven Lehavy, Amy Y. Zang, and Rong Zheng. 2018. Analyst information discovery and interpretation roles: A topic modeling approach. *Management Science*, 64(6):2833–2855.

Allen H. Huang, Amy Y. Zang, and Rong Zheng. 2014. Evidence on the information content of text in analyst reports. *The Accounting Review*, 89(6):2151–2180.

Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. 2023. Financebench: A new benchmark for financial question answering. *ArXiv*, abs/2311.11944.

Soyeong Jeong, Taehee Jung, Sung Ju Hwang, Joo-Kyung Kim, and Dongyeop Kang. 2025. When thoughts meet facts: Reusable reasoning for long-context lms. *CoRR*, abs/2510.07499.

Jia-Huei Ju, Yu-Shiang Huang, Cheng-Wei Lin, Che Lin, and Chuan-Ju Wang. 2023. A compare-and-contrast multistage pipeline for uncovering financial signals in financial reports. In *ACL (1)*, pages 14307–14321. Association for Computational Linguistics.

Roman Koshkin, Pengyu Dai, Nozomi Fujikawa, Masahito Togami, and Marco Visentini Scarzanella. 2025. Margen: Multi-agent LLM approach for self-directed market research and analysis. *CoRR*, abs/2508.01370.

Van-Duc Le. 2024. Auto-generating earnings report analysis via a financial-augmented LLM. *CoRR*, abs/2412.08179.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *ACL*, pages 7871–7880. Association for Computational Linguistics.

Hanglin Lu, Yongjie Zhang, and Jinchang Xu. 2025. Extraction of characteristic information from financial super-long texts and prediction of corporate violations. *Research in International Business and Finance*, page 103079.

R Thomas McCoy, Shunyu Yao, Dan Friedman, Mathew D Hardy, and Thomas L Griffiths. 2024. Embers of autoregression show how large language models are shaped by the problem they are trained to solve. *Proceedings of the National Academy of Sciences*, 121(41):e2322420121.

Nafise Sadat Moosavi, Andreas Rücklé, Dan Roth, and Iryna Gurevych. 2021. Scigen: a dataset for reasoning-aware text generation from scientific tables. In *NeurIPS Datasets and Benchmarks*.

Rajdeep Mukherjee, Abhinav Bohra, Akash Banerjee, Soumya Sharma, Manjunath Hegde, Afreen Shaikh, Shivani Shrivastava, Koustuv Dasgupta, Niloy Ganguly, Saptarshi Ghosh, and Pawan Goyal. 2022. Ectsum: A new benchmark dataset for bullet point summarization of long earnings call transcripts. In *EMNLP*, pages 10893–10906. Association for Computational Linguistics.

John J Murphy. 1999. *Technical analysis of the financial markets: A comprehensive guide to trading methods and applications*. Penguin.

Natapong Nitarach, Warit Sirichotedumrong, Panop Pitchayarthorn, Pittawat Taveekitworachai, Potsawee Manakul, and Kunat Pipatanakul. 2025. FinCoT: Grounding chain-of-thought in expert financial reasoning. In *Proceedings of The 10th Workshop on Financial Technology and Natural Language Processing*, pages 93–123, Suzhou, China. Association for Computational Linguistics.

Ankur P. Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. 2020. Totto: A controlled table-to-text generation dataset. In *EMNLP (1)*, pages 1173–1186. Association for Computational Linguistics.

Joon Sung Park, Joseph C. O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *UIST*, pages 2:1–2:22. ACM.

Stephen H. Penman. 2013. *Financial Statement Analysis and Security Valuation*, 5th edition. McGraw-Hill/Irwin, New York.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2023. Measuring and narrowing the compositionality gap in language models. In *EMNLP (Findings)*, Findings of ACL, pages 5687–5711. Association for Computational Linguistics.

Ratish Puduppully, Li Dong, and Mirella Lapata. 2019. Data-to-text generation with content selection and planning. In *AAAI*, pages 6908–6915. AAAI Press.

Yijia Shao, Yucheng Jiang, Theodore A. Kanell, Peter Xu, Omar Khattab, and Monica S. Lam. 2024. Assisting in writing wikipedia-like articles from scratch with large language models. In *NAACL-HLT*, pages 6252–6278. Association for Computational Linguistics.

Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. 2023. Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In *EMNLP (Findings)*, Findings of ACL, pages 9248–9274. Association for Computational Linguistics.

Anmol Singhal Navya Singhal. 2025. Analogy-driven financial chain-of-thought (ad-fcot): A prompting approach for financial sentiment analysis. *CoRR*, abs/2509.12611.

Takehiro Takayanagi, Tomas Goldsack, Kiyoshi Izumi, Chenghua Lin, Hiroya Takamura, and Chung-Chi Chen. 2025. Earnings2Insights: Analyst report generation for investment guidance. In *FinNLP*, pages 246–251. Association for Computational Linguistics.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *ACL (1)*, pages 10014–10037. Association for Computational Linguistics.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024a. Voyager: An open-ended embodied agent with large language models. *Trans. Mach. Learn. Res.*, 2024.

Zhiruo Wang, Graham Neubig, and Daniel Fried. 2024b. Trove: Inducing verifiable and efficient toolboxes for solving programmatic tasks. In *ICML*, Proceedings of Machine Learning Research, pages 51177–51191. PMLR / OpenReview.net.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent workflow memory. In *ICML*, Proceedings of Machine Learning Research. PMLR / OpenReview.net.

Nirmalie Wiratunga, Ramitha Abeyratne, Lasal Jayawardena, Kyle Martin, Stewart Massie, Ikechukwu Nkisi-Orji, Ruvan Weerasinghe, Anne Liret, and Bruno Fleisch. 2024. CBR-RAG: case-based reasoning for retrieval augmented generation in llms for legal question answering. In *ICCBR*, Lecture Notes in Computer Science, pages 445–460. Springer.

Sam Wiseman, Stuart M. Shieber, and Alexander M. Rush. 2017. Challenges in data-to-document generation. In *EMNLP*, pages 2253–2263. Association for Computational Linguistics.

Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Stuart Rosenberg, and Gideon Mann. 2023. Bloomberggpt: A large language model for finance. *ArXiv*, abs/2303.17564.

Qianqian Xie, Weiguang Han, Xiao Zhang, Yanzhao Lai, Min Peng, Alejandro Lopez-Lira, and Jimin Huang. 2023. PIXIU: A comprehensive benchmark, instruction dataset and large language model for finance. In *NeurIPS*.

Hongyang Yang, Xiao-Yang Liu, and Chris Wang. 2023. Fingpt: Open-source financial large language models. *ArXiv*, abs/2306.06031.

Kevin Yang, Yuandong Tian, Nanyun Peng, and Dan Klein. 2022. Re3: Generating longer stories with recursive reprompting and revision. In *EMNLP*, pages 4393–4479. Association for Computational Linguistics.

Ling Yang, Zhaochen Yu, Tianjun Zhang, Shiyi Cao, Minkai Xu, Wentao Zhang, Joseph E. Gonzalez, and Bin Cui. 2024a. Buffer of thoughts: Thought-augmented reasoning with large language models. In *NeurIPS*.

Yajing Yang, Tony Deng, and Min-Yen Kan. 2025. KAHAN: knowledge-augmented hierarchical analysis and narration for financial data narration. In *EMNLP (Findings)*, pages 25761–25785. Association for Computational Linguistics.

Yajing Yang, Qian Liu, and Min-Yen Kan. 2024b. Datatales: A benchmark for real-world intelligent data narration. In *EMNLP*, pages 10764–10788. Association for Computational Linguistics.

Zhaorui Yang, Bo Pan, Han Wang, Yiyao Wang, Xingyu Liu, Luoxuan Weng, Yingchaojie Feng, Haozhe Feng, Minfeng Zhu, Bo Zhang, and Wei Chen. 2026. Multimodal deepresearcher: Generating text-chart interleaved reports from scratch with agentic framework. In *AAAI*, pages 34368–34377. AAAI Press.

Michihiro Yasunaga, Xinyun Chen, Yujia Li, Panupong Pasupat, Jure Leskovec, Percy Liang, Ed H. Chi, and Denny Zhou. 2024. Large language models as analogical reasoners. In *ICLR*. OpenReview.net.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *AAAI*, pages 19724–19731. AAAI Press.

Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V. Le, Ed H. Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. 2024. SELF-DISCOVER: large language models self-compose reasoning structures. In *NeurIPS*.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *ACL/IJCNLP (1)*, pages 3277–3287. Association for Computational Linguistics.

A Extraction Pipeline Configuration

This appendix details the hyperparameters, prompts, and implementation choices for the four-pass extraction pipeline (§3.2). The pipeline takes a CSV of expert analyst reports with columns `sector`, `symbol`, `company`, `author`, `date`, `url`, `cleaned_text` and writes a SQLite-backed *Analysis* library.

A.1 Hyperparameters

Pass A tags each identified move with one of seven seed types (attribution, derivation, flagging, comparison, projection, gap detection, and stress test) to steer extraction, together with three accompanying fields (`what_analyst_did`, `what_triggered_it`, `result_text`).

Pass	Parameter	Value
Pass A (Induce)	temperature	0.3
	move types	7
Pass B (Generalize)	temperature	0.3
	retry budget	1
Pass C (Deduplicate)	temperature	0.2
	cluster threshold (cosine)	0.88
	max cluster size	30
	cross-batch dedup threshold	0.85
	embedding batch	100
	clustering algorithm	greedy single-linkage
Pass D (Quality-filter)	temperature	0.1
	criteria	3 (transferable, actionable, grounded)
	retry budget	1

Table 5: Extraction pipeline hyperparameters.

A.2 Pass A: Induce

Pass A system prompt

```
You are examining an expert financial analysis report to identify analytical moves — moments where the analyst performed a specific reasoning operation rather than simply describing or summarizing.
An analytical move has three properties:
1. It required the analyst to do something active (compare, derive, decompose, flag, project, contrast)
2. It produced an insight not directly stated in the source material
3. It could in principle be applied to a different company or situation
For each analytical move you find, output:
- "move_type": one word describing the operation [attribution | derivation | flagging | comparison | projection | gap_detection | stress_test]
- "what_analyst_did": one sentence describing the specific operation performed
- "what_triggered_it": what data condition or pattern in the source material prompted this move
- "result_text": the exact span in the report that is the output of this move
Do not output moves that are simple description or summary. Output only moves that are non-generic and could transfer to a different earnings report. Output only valid JSON, no commentary.
```

Pass A issues one LLM call per report. The system prompt frames the task around three properties of an analytical move (active reasoning, non-trivial output, transferability) and requests a JSON list of moves. Malformed items missing any of the four required keys are silently dropped. The output is a per-report list of candidate moves; reports yielding no moves are skipped.

A.3 Pass B: Generalize

Pass B issues one LLM call per candidate. The system prompt requires a JSON object with exactly the three fields used by the rest of the pipeline (`data_signal`, `analytical_move`, `reference_text`) and includes an explicit transferability test ("would this description match the same situation in a hospital, a retailer, and a defense contractor?"). The pass is callable with a `retry_instruction` that is appended to the system prompt; this slot is used by Pass D to request targeted re-generation when a *Analysis* fails the transferability or actionability check.

Pass B system prompt

```
You are converting a specific analytical move from one expert report into a reusable Analysis applicable to any earnings report.
Output a JSON object with three fields:
- "data_signal": the data pattern that should trigger this Analysis. Written at the causal pattern level. No company names, specific numbers, or industry terminology. A reader should recognize this pattern in a report about any company in any sector.
- "analytical_move": what to do when this pattern is found. Written as a clear imperative. Specific enough that two analysts following it independently would produce similar outputs.
- "reference_text": copied verbatim from result_text in the input.
Test for data_signal quality: would this description match the same situation in a hospital, a retailer, and a defense contractor? If yes, the abstraction is correct. If it contains domain-specific terms, abstract further.
Output only valid JSON, no commentary.
```

Source metadata is attached deterministically after Pass B completes: `sector`, `symbol`, `company`, `date`, `url`, and `author` are copied from the CSV row, and the `reference_text` is wrapped with a trailing source label "— <company> <date>" before being stored as the first element of the `reference_texts` list.

Retry instructions used by Pass D.

transferability retry

```
The data_signal is too domain-specific. Abstract it further so it would match the same causal pattern in a hospital, a retailer, and a defense contractor. Remove all industry terminology and specific numbers.
```

actionability retry

```
The analytical_move is too vague. Replace any phrase like "analyze X" with specific operations: decompose, quantify, attribute, flag, or project. Name what to decompose into, what to quantify, and what to compare against.
```

A.4 Pass C: Deduplicate

Pass C has two stages: a deterministic clustering step on embeddings and an LLM merge step per cluster. Candidates are embedded by their `data_signal` using `text-embedding-ada-002` in batches of 100. Clustering is greedy single-linkage on the unit-normalized embeddings: each candidate is assigned to the first existing cluster whose centroid has cosine similarity ≥ 0.88 with the candidate; otherwise a new cluster is opened. Centroids are running means re-normalized after each addition. Clusters larger than 30 candidates are split into chunks of 30 before being sent to the LLM merge step, to keep prompts within context.

The LLM merge step receives the cluster as a JSON payload of (`data_signal`, `analytical_move`, `reference_texts`) triples and is asked to return a merged *Analysis* per cluster. Reference texts are taken verbatim from the originating candidates rather than from the LLM output, since the LLM was observed to occasionally paraphrase what the prompt required to be verbatim spans. Provenance for a merged *Analysis* is reconstructed by matching the merged `data_signal` back to the originating candidates and aggregating per-field via a scalar-or-list rule (single distinct value $\rightarrow$ scalar; multiple $\rightarrow$ list).

For incremental updates against an existing library, Pass C runs a cross-batch deduplication step before clustering: each new candidate is compared by cosine similarity against existing *Analysis* embeddings, and candidates above the cross-batch threshold of 0.85 are paired with their nearest existing *Analysis* and sent through Pass C as a targeted merge that preserves the existing *Analysis*'s ID (so downstream stores update rather than insert).

Pass C merge prompt

```
You are given a list of candidate Analyses. Identify groups of Analyses that represent the same analytical operation at the pattern level, even if their reference_texts come from different reports or industries.
For each group:
- Keep the data_signal that is most general while still being specific enough to trigger reliably
- Keep the analytical_move that is most complete and specific
- Combine all reference_texts into an array — multiple examples strengthen the validation anchor. Each entry should end with a source label e.g. "— Boeing Q3 2022"
Return Analyses that do not match any other as singletons. Output the merged Analysis library as a JSON array. Each element must have exactly these fields: data_signal, analytical_move, reference_texts. Output only valid JSON, no commentary.
```

A.5 Pass D: Quality-filter

Pass D system prompt

```
Evaluate this Analysis against three criteria. For each, output "pass" or "fail" and one sentence of reasoning.
TRANSFERABLE: would data_signal trigger on the same pattern in a different industry? Fail if it contains domain-specific terms.
ACTIONABLE: if two analysts followed analytical_move independently, would they produce similar outputs? Fail if the action is vague (e.g. "analyze X" rather than "decompose X into Y and Z").
GROUNDED: does reference_text show a real instance of the pattern and its output, not just a description of what the pattern is?
Output JSON with fields: transferable, actionable, grounded, each containing "result" (pass|fail) and "reason" (string).
A Analysis must pass all three to enter the library.
```

Pass D issues one LLM call per *Analysis* asking for a pass/fail verdict on each of three criteria. The criteria are evaluated independently, but the routing of failures is asymmetric:

- **Grounded fail** $\rightarrow$ discard. A *Analysis* whose reference span does not show the pattern executed cannot be repaired by re-generation, so it is removed from the library.
- **Transferable fail** $\rightarrow$ retry Pass B with the transferability retry instruction (above). The originating moves are re-sent through Pass B with the appended instruction, re-merged via Pass C, and re-evaluated. If the retry passes all three criteria, the resulting *Analysis* enters

the library with the original ID preserved; otherwise it is discarded.

- **Actionable fail** → retry Pass B with the actionability retry instruction. Same loop as above.

Parse failures on the Pass D response are treated conservatively as pass-all rather than discard-all, on the principle that an inability to judge a *Analysis* is not evidence against it; this affects $< 1\%$ of *Analyses* in practice.

A.6 Concurrency and Caching

The pipeline parallelizes at two levels: rows of the input CSV are processed in parallel (one thread per row), and within each row Pass B calls are issued in parallel across the moves Pass A identified. All LLM-call results, embeddings, and intermediate candidates are written to a SQLite store keyed by *Analysis* ID and embedding model identifier, so re-runs of the pipeline skip already-extracted *Analyses* and already-embedded items. Embeddings for clustering and for cross-batch dedup share the same cache.

B Bank Details

We characterize each library along two axes, using one keyword taxonomy per axis, since signals are noun-phrase data conditions and moves are verb-phrase operations. Each taxonomy is a hand-built, ordered list of categories defined by keyword patterns over the `data_signal` and `analytical_move` fields, refined against the libraries until coverage stabilized; we assign each *Analysis* to the first matching category (single-label) and collect non-matches as *other*. Figures 6 and 7 show the full signal-by-move distribution for both corpora.

C Narration Pipeline Configuration

This appendix details the hyperparameters (Table 6), prompts, and implementation choices for the four-stage narration pipeline (§4). The pipeline takes a source input (an earnings call transcript with sector tag, or a structured OHLCV DataFrame with a report date and market tag) and produces an analyst-style report.

C.1 Stage 1: Signal extraction

The pipeline supports two signal-extraction backends, selected by input modality. Both produce the same output type: a list of `(type, fields, span)` tuples that drive Stage 2 retrieval.

Stage	Parameter	Value
Stage 1 (signal extraction)		
Earnings (LLM)	temperature	0.2
	mode	hierarchical (default)
	signal types	17 (14 presence + 3 absence)
	per-type cap	6–12 (see Table 7)
Earnings (validation)	temperature	0.1
	error classes	6
DataTales (pandas)	lookback windows	1d, 5d, 20d
	momentum threshold	0.5%
	vol spike / collapse	1.5× / 0.5× hist
	volume spike threshold	2.0× 20d avg
	MA threshold	0.5% from 20d MA
Stage 2 (retrieval)		
Top-k	per input	5
Retrieval mode	default	per-type
Retrieval backend	default	`text-embedding-ada-002`
Sector boost	multiplicative	1.2× on match
Similarity threshold		0.0 (no floor)
Triggers per *Analysis*		up to 2
*Stage 3 (per-*Analysis *analysis)*		
Temperature		0.4
Mode	default	pattern
Parallelism		one thread per *Analysis*
Stage 4 (composition)		
Temperature		0.5
Mode	default (Earnings)	signals (transcript-free)
	default (DataTales)	findings + signals + summary
Validate-and-retry		
Temperature		0.1
Quality verdicts		high / partial / missing
Retry trigger		quality = missing
Retry budget		1

Table 6: Narration pipeline hyperparameters.

Stage 1 prompt (one call per signal type)

```
You are a financial signal extractor.
Extract signals of exactly ONE type from
an earnings transcript.
SIGNAL TYPE: {signal_type}
REQUIRED FIELDS: {fields}
EXTRACTION RULE: {rule}
OUTPUT FORMAT: Return a JSON array. Each
element must have EXACTLY these fields: {
"type": "{signal_type}", "fields": { <the
required fields above, omit fields with
no evidence> }, "span": "<shortest exact
verbatim quote from the transcript, 10-40
words, that supports this signal>" }
Rules: - Extract EVERY distinct instance
of this signal type — do not stop at one.
- span must be copied verbatim from the
transcript. - For ABSENT types: span is the
quote containing the claim/indicator that
reveals the absence. - Do not infer fields
not supported by the transcript text. -
Return [] if no signals of this type are
present. - Output only a valid JSON array,
no commentary.
```

Earnings transcripts (LLM-based). For transcripts, an LLM extracts 17 signal types: 14 *presence* signals (margin delta, incremental margin, guidance revision, volume trend, market share, pricing realization, FX impact, forward signal, cash flow, capital allocation, earnings quality, balance sheet, segment mix, management tone) and 3 *ab-*

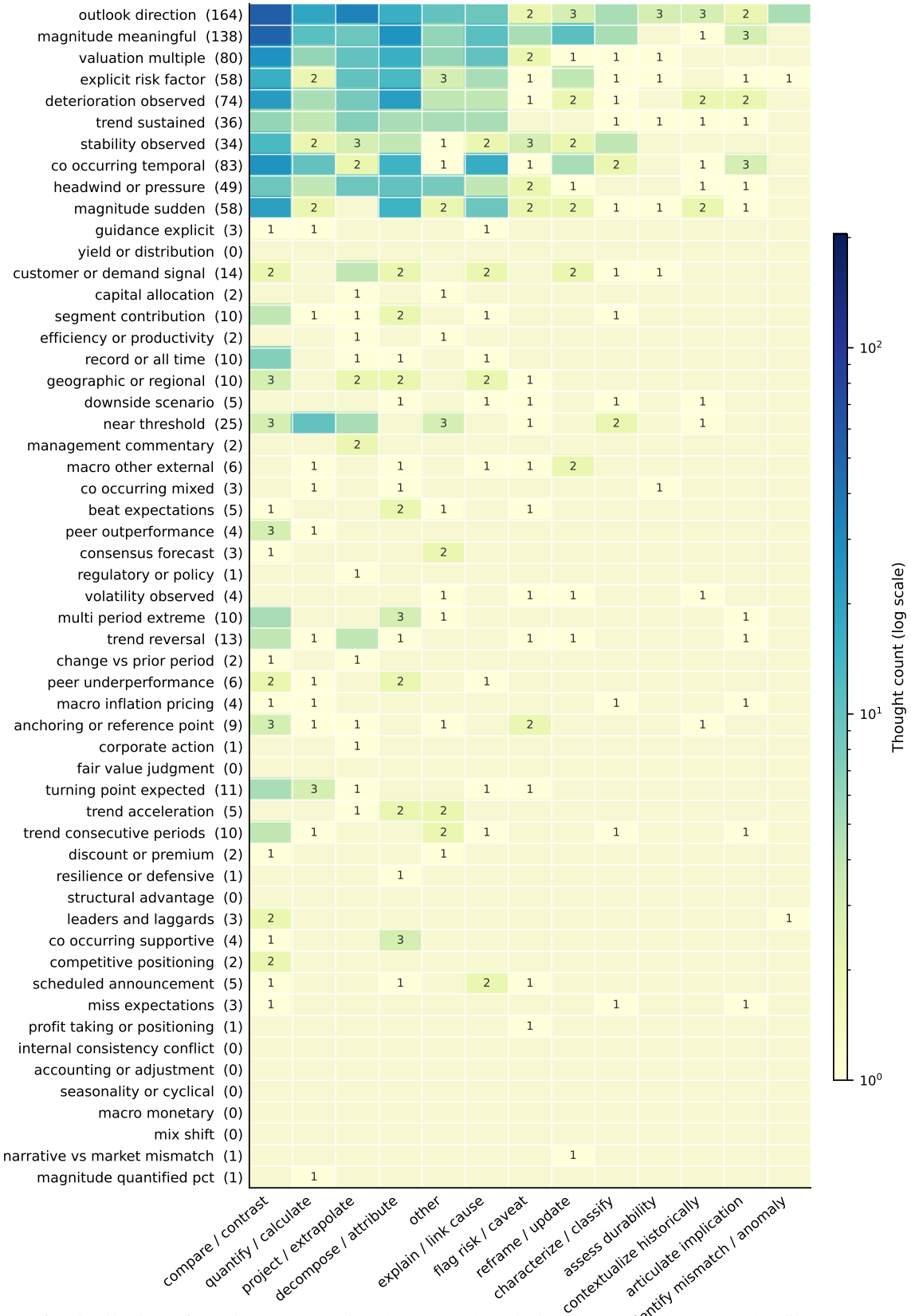


Figure 6: Distribution of *Analyses* across signal types and analytical moves in the DataTales library (n=1,422). Rows are signal types and columns are analytical move types, both ordered by total frequency across the two libraries so that this figure and Figure 7 share axes and color scale. Cell color encodes *Analysis* count on a log scale; sparse cells (counts 1–3) are annotated with their value, and absent pairs are shaded grey. Numbers in parentheses are per-library signal totals.

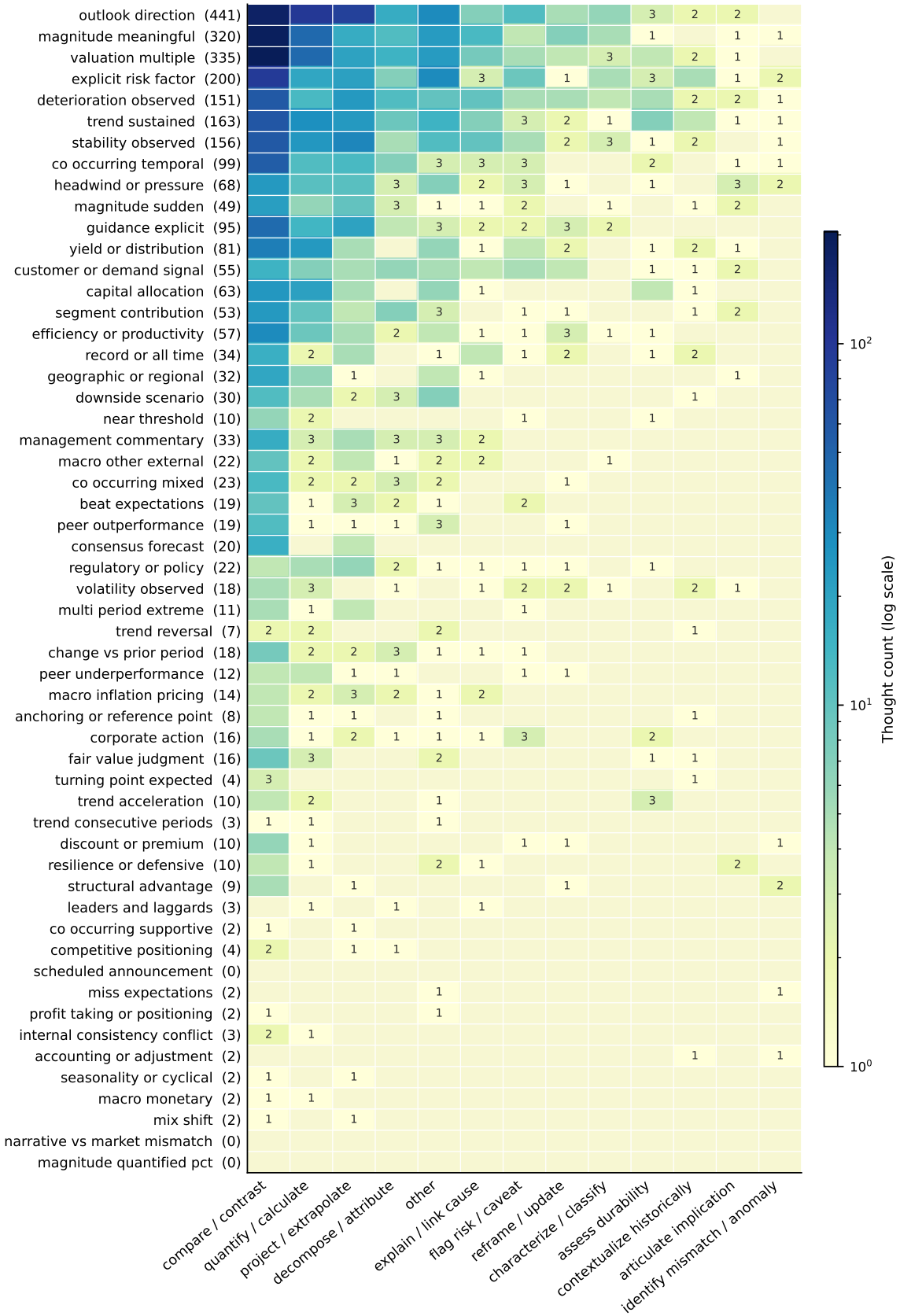


Figure 7: Distribution of *Analyses* across signal types and analytical moves in the Earnings library (n=3,889). Axes, ordering, and color scale are shared with Figure 6. Both libraries concentrate on the same high-frequency signal-move combinations while exhibiting a long tail of rare but distinct analytical patterns.

sence signals (absent mechanism, absent estimate, absent conversion). The hierarchical mode (default) issues one focused LLM call per signal type and runs them in parallel; each call is gated by a keyword-evidence check (e.g. the `MARGIN_DELTA` call fires only if "margin", "basis point", "bps", or "profitability" appears in the transcript), with absence types having empty keyword lists so they always fire.

Per-type caps and numeric prioritization. Hierarchical extraction is permissive (the per-type prompt asks for *every* distinct instance), so each type has a post-extraction cap to prevent any one type from saturating the slate. Within a capped type, signals with non-empty numeric fields (e.g. `magnitude_pct` for `VOLUME_TREND`, `delta_bps` for `FX_IMPACT`) are kept first; non-numeric signals fill remaining slots.

Type	Cap	Type	Cap
GUIDANCE_REVISION	12	MGMT_TONE	8
MARGIN_DELTA	12	PRICING_REALIZATION	8
VOLUME_TREND	10	SEGMENT_MIX	8
FX_IMPACT	10	BALANCE_SHEET	8
MARKET_SHARE	10	EARNINGS_QUALITY	8
FORWARD_SIGNAL	10	ABSENT_ESTIMATE	8
CAPITAL_ALLOCATION	10	ABSENT_CONVERSION	6
ABSENT_MECHANISM	10	CASH_FLOW	6
		INCREMENTAL_MARGIN	6

Table 7: Per-type signal caps in hierarchical extraction.

Fact validation. After extraction, every signal is passed through a fact-validation LLM call that checks six error classes against the verbatim supporting span: *geo_scope* (region-specific figure attributed to the global segment), *period_scope* (full-year forecast tagged as a quarterly actual), *metric_identity* (segment-level figure labelled company-wide; guidance figure labelled actual), *composite_split* (combined dividend + buyback amount assigned to one action), *organic_vs_reported* (reported growth labelled organic), and *adjusted_vs_gaap* (non-GAAP figure labelled GAAP). Errors with a confident correction are applied in place; errors without are flagged. The validation prompt is conservative (only flags errors with clear span evidence), and parse failures are treated as pass-through.

DataTales (deterministic). For structured market data, signals are computed directly from OHLCV series in pandas with no LLM involvement. The extractor produces eight signal types: `PRICE_MOMENTUM` (1d/5d/20d returns above a 0.5% threshold), `VOLATILITY_SIGNAL` (annualized 20d realized vol versus 60d rolling baseline, classified as spike / elevated / normal / suppressed), `VOLUME_ANOMALY` (today's volume versus 20d average), `TREND_SIGNAL` (close versus 20d MA), `TERM_STRUCTURE` (front-vs-second-month spread for futures markets), `CONTRACT_ROLL` (5d-average volume crossover), `YIELD_CURVE` (2s10s, 2s30s, 3m10s spreads for treasury markets), and `CROSS_ASSET` (VIX regime and 10Y vs equities for equity markets, USD direction for currencies, Brent-WTI spread for energy). For the equity market only, per-product signals are restricted to a set of primary instruments (S&P 500, Nasdaq Composite, Nasdaq 100, Dow Jones, Russell 2000, VIX, US 10-Year Bond Yield) to prevent the slate from being saturated by individual constituents.

Signal validation (span check). For LLM-extracted signals, the supporting span is verified against the transcript via whitespace-normalized substring match; signals whose span cannot be located are flagged `span_hallucinated` and dropped. `MARGIN_DELTA` cause-types are additionally checked against the supporting span; unsupported causes are stripped from the signal rather than flagging it whole.

C.2 Stage 2: Retrieval

Stage 2 retrieves a slate of $k = 5$ *Analyses* from AnalysisBank given the signal list. Three retrieval modes and three backends are supported, with the defaults indicated below.

Embedding target. The default retrieves by cosine similarity between signal descriptions and the `data_signal` field of each *Analysis*. An ablation retrieves against `reference_texts` (concatenated with trailing source attributions stripped); this variant is reported in §6.1.

Sector boost. Each *Analysis*'s similarity score is multiplied by 1.2 if the *Analysis*'s source sector matches the input's; sector membership is checked as exact match or as set-membership when a *Analysis* has multiple source sectors. The boost favors in-sector *Analyses* without hard-filtering out cross-sector ones, since some analytical moves transfer across sectors.

Retrieval mode. The default is *per-type* retrieval: each fired signal type contributes one representative signal (selected by largest numeric magnitude using a per-type priority list — `delta_bps` for

margin signals, `magnitude_pct` for volume, both `old_guidance` and `new_guidance` for guidance, etc.), and the best *Analysis* for that representative is added to the slate. Types are processed in order of representative-signal significance, and remaining slate slots are filled by global cosine top-k to avoid leaving the slate short when fewer than k signal types fire. The alternative *cosine* mode applies global top-k directly without per-type structure.

Retrieval backend. The default backend embeds signals and *Analyses* with OpenAI's `text-embedding-ada-002`. Two alternatives are available for ablation: BM25 over tokenized signal and *Analysis* text, and a local sentence-transformer (`all-MiniLM-L6-v2`). Backend choice is independent of retrieval mode (e.g. BM25 + per-type is valid). All backends apply the same 1.2× sector boost and the same optional similarity-threshold cutoff (default 0, i.e. no filtering on raw similarity).

Trigger mapping. For each *Analysis* in the final slate, the top-2 signals by cosine similarity to the *Analysis* embedding are tagged as the *Analysis*'s *triggering signals*; their verbatim spans are passed to Stage 3 alongside the *Analysis*.

Embedding cache. *Analysis* embeddings are computed once and stored in a SQLite cache keyed by *Analysis* ID and embedding model. The default `data_signal` embeddings and the reference-text variant each live in their own cache table, so switching ablation variants does not require re-embedding.

C.3 Stage 3: Per-*Analysis* analysis

For each *Analysis* in the Stage 2 slate, an independent LLM call applies the *Analysis*'s `analytical_move` to its triggering signals and supporting spans (transcript-side) or to the market data context (DataTales). The prompt forbids section headers and framing language, because Stage 4 imposes structure. Calls are issued in parallel across *Analyses* via a thread pool.

The user prompt assembles three blocks: `PATTERN`, the `analytical_move` to execute; `SIGNALS`, the triggering signals serialized as JSON (`type` and structured `fields`); and `EXCERPTS`, their verbatim supporting spans.

Stage 3 prompt

```
You are a financial analyst performing a
specific analysis.  Apply the analytical
pattern to the provided signals and
transcript excerpts.  Be precise and
quantitative where the data supports it.
Do not write section headers or introductory
framing. Your output will be combined with
other analyses into a broader report.
—
PATTERN: {analytical_move}
SIGNALS: [{"type": ..., "fields": {...}},
...]
EXCERPTS: "{verbatim supporting span}" ...
```

Three reference modes (ablation). The default mode (`pattern`) sends only the pattern, signals, and spans, omitting the *Analysis*'s `reference_text`. The `pattern_with_refs` mode additionally includes the reference text(s) as a depth and tone anchor, inserted as a `REFERENCE EXAMPLES` block between `PATTERN` and `SIGNALS`. The `refs_only` mode passes only the reference text(s) and signals, without the pattern — testing whether reference examples alone are sufficient. The ablation isolates which component of the three-field representation carries the analytical depth gain (§6.1).

DataTales adaptation. For DataTales, Stage 3 uses a market-specific system prompt that forbids corporate-earnings language ("management", "guidance", "EPS") and asks for prices, spreads, and time-window references instead. Triggering signals are still the top-2 by cosine, but the supporting context is the deterministic market data summary (§C.6) rather than transcript spans.

C.4 Stage 4: Composition

A single LLM call composes the per-*Analysis* findings into a final report. The prompt enforces three structural constraints: an executive summary (3–4 sentences naming takeaway, bull case, and key risk), theme sections with analyst-chosen titles drawn from the evidence, and an investment-recommendations block with horizon-differentiated calls (Next Day / Week / Month) each tagged Long / Short / Neutral with a one- to two-sentence rationale. The prompt directs the writer to treat findings as the analytical backbone and to use signals only for factual gap-filling.

Stage 4 prompt

```
You are writing an earnings analysis report. The following analytical findings have been derived from the earnings call. Compose them into a coherent report organised around investment-relevant themes that cut across multiple metrics (e.g., demand environment, margin dynamics, capital allocation, guidance credibility, regional mix, risk factors). Let the data shape the themes — use as many sections as the story requires.
Do not repeat analysis — synthesize and connect where findings relate to each other. Use the signals to fill any factual gaps the findings do not cover.
REPORT STRUCTURE:
**Executive Summary** (3-4 sentences) State the single most important takeaway, the dominant bull case, and the key risk. This should read as the opening of a research note — sharp and investment-actionable.
**[Theme sections — analyst's choice of titles]** Each section covers a cross-cutting investment theme. For each: - Lead with the analytical insight that frames the theme. - Weave in specific numbers, percentages, and period comparisons from the relevant findings. - Where evidence from multiple findings reinforces or conflicts with each other, integrate it. - Close each section with the forward implication for the investment case.
**Investment Recommendations** After synthesising the full picture, provide differentiated calls across horizons. Each horizon reflects a distinct lens — do not recycle the same rationale: - **Next Day:** Headline beat/miss reaction and any guidance surprise. - **Next Week:** Sector/macro regime and near-term catalysts or overhangs. - **Next Month:** Fundamental execution quality — margins, cash flow, order book durability. For each horizon state: **Long / Short / Neutral** and a 1-2 sentence rationale naming the specific data points that drove the call.
```

Input variants. Four prompt variants are used depending on what context Stage 4 receives:

- *Default (findings + signals)*: the production configuration. Signals fill factual gaps the findings do not cover.
- *With transcript*: additionally supplies the raw transcript or market data summary, with a primary rule that the findings still drive structure and the transcript only fills coverage gaps. Used when the deployment can tolerate the additional input length.
- *Transcript-only / signals-only / findings-only*: ablations that strip the input progressively, used in the component analysis in §6.1.

For DataTales, the same four variants exist with adapted system prompts that swap earnings vocabulary for market-data vocabulary and that ask for trading-relevant horizon calls (momentum continuation, technical structure, term-structure outlook) rather than equity-analyst horizons.

DataTales context augmentation. On DataTales, Stage 4 prompts may optionally include a *per-entity raw OHLCV block* (the last 20 rows for each product on or before the report date). This is included automatically in the default mode and in the signals-only ablation when a DataFrame is provided to the pipeline. It is not used in the transcript-equivalent earnings flow.

C.5 Validate-and-retry

After composition, a per-*Analysis* validator checks whether each *Analysis*'s analytical pattern was applied in the report. The validator issues one LLM call per *Analysis* in the slate (in parallel) and emits a JSON verdict with `applied` (boolean), `quality` ("high", "partial", or "missing"), and a one-sentence note. Verdicts of `missing` trigger a targeted re-run: Stage 3 is re-executed for only the missing *Analyses*, the updated findings are merged into the existing findings list, and Stage 4 is re-composed. The loop is bounded by a retry budget (default 1) and terminates as soon as no missing verdicts remain.

The validator uses the same backbone LLM as the rest of the pipeline. Parse failures default to `missing=False, quality="missing"`, which is conservative in the sense that it does not trigger spurious retries but does flag the *Analysis* as not validated.

C.6 Market data summary (DataTales only)

For DataTales inputs, a deterministic summarizer converts the OHLCV DataFrame into a human-readable context block used by Stages 3 and 4. For each product on or before the report date, the summary emits: last close, 1d/5d/20d returns, position relative to the 20d moving average, annualized 20d realized volatility, and volume ratio against the 20d average. For futures markets, a term-structure line is appended (front vs. second-month spread with backwardation/contango label). For the equity market, primary instruments get a full paragraph each; secondary products are collapsed into a one-line "sector / other products" summary capped at ten entries.

C.7 Concurrency and caching

Stage 1 hierarchical extraction, Stage 3 per-*Analysis* analysis, and validate-stage checks all use thread pools sized to the number of items processed (one thread per signal type, per *Analysis*, or per validation call). LLM responses are cached at the level of `(prompt, system_prompt, temperature)` tuples; cached calls are served without an API hit, which lets ablations re-use intermediate results across configurations that share earlier stages. *Analysis* embeddings are persisted to SQLite as described in §C.2.

The full default configuration on Earnings2Insights requires roughly $1 + 17 + 1 + 5 + 1 + 5 = 30$ LLM calls per report (1 if hierarchical mode condenses to a single call, 17 for the per-type fan-out, 1 for fact validation, 5 for Stage 3, 1 for Stage 4, 5 for the validator) before any retries; on DataTales it is $5 + 1 + 5 = 11$ since Stage 1 is deterministic. Retries add up to $5 + 1$ calls per cycle, bounded by the retry budget.

D Structure-Level Baseline Adaptation

Buffer of Thoughts (BoT) (Yang et al., 2024a) and Agent Workflow Memory (AWM) (Wang et al., 2025) were originally evaluated on math reasoning and web-agent trajectories respectively. We adapt both to analytical report generation while preserving their native abstraction granularity (one full-report template or workflow per task), so that the comparison with ANALYSISBANK (§5) isolates decomposition granularity as the variable under test. Both methods are induced from the same 550-report corpus that ANALYSISBANK consumes, with the same backbone model for induction and inference.

D.1 Agent Workflow Memory

AWM induces reusable workflows from training trajectories and injects the relevant workflow into the inference-time prompt.

Trajectory representation. Each expert report in the induction corpus is decomposed into a sequence of `(observation, action)` pairs by splitting on section headings (markdown headings, bold headers, or colon-terminated section labels). Each pair represents one reasoning step the analyst took: the heading is the `observation` (the context the analyst was about to write into), and the section body is the `action` (what was written). Reports without recognisable section structure are split into fixed 400-character chunks. This mirrors AWM's original trajectory format from web-agent execution traces.

Offline induction. Workflows are induced per GICS sector (eleven sectors for Earnings2Insights; per-market for DataTales). For each sector, up to ten training reports are sampled, decomposed into trajectories, and passed to an LLM with an induction prompt that asks for a 5–10 step numbered workflow capturing the analytical pattern shared across the reports. The induction prompt enforces abstraction ("do not reproduce specific figures from these reports") and reusability ("apply to any future report for this ticker/sector"). One workflow per sector is stored as a plain-text file in a sector-keyed directory.

Inference-time retrieval. For a new input with ticker t in sector s, the workflow is resolved by a four-step fallback: (i) exact ticker match, (ii) exact sector match, (iii) LLM-selected closest peer from same-sector candidates, (iv) LLM-selected cross-sector fallback. Step (iii) uses a structured JSON prompt over a ranked menu of candidate workflows; step (iv) is reached only when sector metadata is unavailable. In our benchmark configuration, step (ii) resolves every test instance, so the LLM-selection fallback is rarely exercised.

Inference-time generation. The selected workflow is injected at the top of the generation prompt as an "Analyst Workflow (induced from past reports)" block, followed by the task description, the source data, and a request to "follow the workflow above as a guide for structuring your analysis [and] adapt each step to the current data." One LLM call produces the final report. Crucially, the workflow is the only retrieved artifact: AWM does not extract typed signals, does not retrieve multiple patterns, and does not run per-pattern execution before composition.

D.2 Buffer of Thoughts

BoT maintains a meta-buffer of structured "thought templates" and applies a four-step inference pipeline (problem distillation → buffer retrieval → template instantiation → reasoner instantiation) per task. We use the canonical 5-section template format from the original paper: *Key Information*, *Domain Constraints*, *Abstract Task Description*, *Python Logic*, and *Answer Format*.

Offline induction. Templates are induced per training report rather than per sector. For each report in the induction set (sampled at ten per sector to match AWM's induction budget), the report is passed to an LLM with a thought-distillation prompt that asks for a 5-section template covering the analytical pattern the report instantiates, with all company-specific figures abstracted away. The resulting template is compared against the closest existing template in the buffer via an LLM-judged novelty check; templates judged novel are added, near-duplicates are skipped. The novelty filter prevents the buffer from accumulating paraphrases of the same template while still allowing genuinely distinct analytical patterns to coexist. Templates are stored in a persistent ChromaDB collection with sentence-transformer embeddings (all-MiniLM-L6-v2) of the template text.

Inference-time retrieval. For a new input, an LLM first performs *problem distillation*: it produces a 4-section distillation of the input (Key Information, Domain Constraints, Abstract Task Description, Answer Format) that serves as the retrieval query. Cosine similarity against the meta-buffer returns the single closest template. The retrieved template is then *instantiated* by a second LLM call, which adapts each abstract section to the current company, period, and data, producing a problem-specific briefing.

Inference-time generation. The instantiated template is injected at the top of the generation prompt as an "Analyst Briefing (Buffer of Thought)" block, followed by the task description and the source data. One LLM call produces the final report. As with AWM, the template is the only retrieved artifact: a single top-down structure covers the whole report, and no per-move retrieval or per-pattern execution occurs.

Online buffer update. The original BoT formulation supports an optional online update that distils each newly produced report back into a template and adds it to the buffer if a novelty check passes. We disable this in the evaluated configuration to keep the buffer fixed and comparable to AWM's offline-only induction.

D.3 Why this adaptation is fair

We preserve each method's native abstraction granularity so that decomposition granularity is the variable under test. Adapting BoT or AWM to retrieve at the move level would convert them into variants of ANALYSISBANK rather than independent baselines. We do adapt both to consume the same induction corpus as ANALYSISBANK (§3.2), so any performance gap reflects how the corpus is decomposed, not what it contains. The resulting contrast is explicit: at induction, each report yields one workflow or template versus the multiple analytical moves ANALYSISBANK extracts; at inference, one structure is retrieved top-down versus the slate of move-level patterns ANALYSISBANK retrieves and composes.

E Metric

This appendix specifies the judge models, the per-metric judgment protocols, and the design choices that bear on reliability.

E.1 Judge models

Two LLM judges are used in the evaluation pipeline:

- **Theme extractor (Claude Sonnet 4.6).** Reads each source input (transcript or market-data summary) once, independently of any generated report, and emits the list of *material themes* — themes a competent analyst would be expected to address. The same theme list is then reused across every model variant evaluated on that input, so coverage scoring is anchored to a fixed reference rather than re-elicited per condition.
- **Per-claim and per-report judge (Gemini 3 Flash).** Runs claim extraction (Stage 1), reasoning-depth scoring (Stage 2a), data-analytic-style scoring (Stage 2b), theme-coverage matching, factual scoring, and head-to-head preference scoring against expert references. Every judgment uses temperature 0.0 and structured JSON output, with up to three retries on JSON-parse failure.

Both judges are different model families from any of the four evaluation backbones (Qwen3-8B, Qwen3.5-9B, DeepSeek-V4-Flash, GPT-5.1) to reduce self-preference bias in line with recent LLM-as-judge findings.

E.2 Per-metric protocols

%insight and %analysis. The judge first extracts analytical claims from the report in 80-character-minimum chunks split on paragraph

breaks, with boilerplate (executive summary, investment-recommendations sections) stripped before extraction. Each claim is then labelled in one of four categories:

- **factual**: depth-1 claims that restate a number or fact from the source.
- **novel**: a data-specific insight that combines conditions, attributes causes, or projects consequences in a way not directly stated in the source.
- **standard**: an analytical claim that a competent analyst would routinely make from this data.
- **generic**: a claim that applies to any report of this type and does not depend on the specific data.

%insight is novel/(novel + standard + generic + factual); %analysis adds standard to the numerator.

depth. For each extracted claim the judge counts *analytical hops* — distinct inference steps needed to derive the claim from the source — and emits the chain of hop types (e.g. `FACTUAL`, `COMPARE`, `ATTRIBUTE`, `PROJECT`). depth is the mean hop count per claim. Depth-1 claims are auto-assigned the *factual* label and skipped from analytical type scoring, so the two metrics share a single claim-extraction pass and are not double-counted.

%factual. Numerical values in the report are checked sentence-by-sentence against the source. For Earnings2Insights the reference is the transcript itself: for each report sentence containing a financial number, candidate reference sentences are selected from the transcript by content-token overlap (stopwords stripped), and the judge labels each numerical value as `CORRECT`, `INCORRECT`, or `DONT_KNOW`. For DataTales the reference is a structured table of pre-computed values (OHLCV plus derived metrics: 1d/5d/20d returns, 20d moving average, annualized volatility, volume ratio, term-structure spread); every report sentence is checked against the full table without a token-overlap filter, since every entry is potentially relevant. Date/time tokens (years, quarters, ISO dates, day-of-month integers adjacent to month names) are filtered out before scoring so they do not contaminate the financial-value count. %factual is correct/(correct + incorrect) over the full report.

	%novel	%analysis
Human rate: ANALYSISBANK (%)	15.3	51.0
Human rate: baseline (%)	1.7	25.4
Δ (pp) [95% CI]	+13.7 [8.7, 18.3]	+25.6 [17.8, 33.7]
Report-level Pearson r	0.64	0.65
Claim-level κ (inter-annotator)	0.25	0.49
Full-taxonomy κ: LLM judge–annotator	0.41, 0.47	
Full-taxonomy κ: inter-annotator	0.50	

Table 8: Claim-level human study (277 claims, blind to system and LLM judge labels). Claim-level κ is inter-annotator agreement on each metric's cut (novel vs. rest; novel+standard vs. rest); full-taxonomy κ gives agreement on the full taxonomy.

%win. Each model report is judged head-to-head against the expert reference (Seeking Alpha analyst reports for Earnings2Insights, the dataset's human-written market reports for DataTales) by two personas — *analyst* and *investor* — each with its own system prompt and rubric. To control for position bias, each persona judges the pair under both orderings (model first, expert first), yielding $2 \times 2 = 4$ judgments per report. The two ordering outcomes are aggregated per persona by majority: two agreeing outcomes set the persona's verdict; *model+tie* resolves to model and *expert+tie* to expert, treating ties as the weaker signal in either direction; *model+expert* resolves to tie. %win is reported per persona and as the mean across personas.

E.3 Reliability checks

To test the claim-level labels against human judgment, two human annotators applied the LLM judge's claim taxonomy (novel/standard/generic/-factual) to 277 claims (10 instances, ANALYSISBANK and the best baseline), blind to both system identity and the LLM judge's labels. Table 8 reports the results. Annotators mark ANALYSISBANK claims as novel significantly more often than baseline claims (15.3% vs. 1.7%) and as analytical significantly more often (51.0% vs. 25.4%), and human and judge per-report rates correlate for both metrics (Pearson r=0.64 and 0.65). On the full-taxonomy taxonomy, the judge agrees with each annotator (κ=0.41 and 0.47) at a level comparable to the annotators' agreement with each other (κ=0.50). Agreement is lowest on the novel-vs.-rest cut (κ=0.25): annotating a single claim as novel is subjective, so we treat the report-level rates as the unit of evaluation.

Benchmark	Comparison	Win	Loss	Win%
E2I	AnalysisBank vs Direct	29	1	96.7
E2I	AnalysisBank vs BoT	30	0	100.0
DataTales	AnalysisBank vs CoT	26	4	86.7
DataTales	AnalysisBank vs BoT	19	11	63.3

Table 9: Human pairwise preferences (consensus across two annotators, n=30 votes per comparison). No ties were recorded.

F Human Evaluation Details

F.1 Insight quality

Sampling. For each benchmark, we rank all Qwen3-8B test instances by the insight-rate gap between AnalysisBank and the best baseline, then stratify-sample 5 instances from each tercile (top, middle, bottom), yielding 15 sets per benchmark (30 total). Each set contains three reports: AnalysisBank, the best prompting baseline (CoT for DataTales, Direct for Earnings2Insights), and BoT.

Protocol. Two annotators judge each set. Reports are labelled A, B, C with method identity hidden and presentation order randomized per set. Annotators rank the three reports from 1 (most insightful) to 3 (least insightful), with no ties allowed. The instruction defines insight as "substantive, non-obvious analytical content—e.g., specific causal explanations, comparisons, projections, or implications grounded in the input—rather than generic commentary, restated facts, or boilerplate language."

Results by benchmark. Table 9 reports consensus pairwise outcomes (both annotators combined).

Inter-annotator agreement. Table 10 reports inter-annotator agreement with bootstrap 95% CIs (N=30). Annotators agree on the top-ranked report in 66.7% of sets (chance: 33.3%) and on the pairwise preferences in 90.0% (vs. prompting) and 70.0% (vs. BoT) of comparisons; agreement on the middle rank (36.7%) is near chance. Cohen's κ (0.37 vs. prompting; 0.13 vs. BoT) is lower than raw agreement because AnalysisBank wins most comparisons and κ is deflated under skewed label distributions (Feinstein and Cicchetti, 1990).

F.2 Factuality

We complement %factual with a human evaluation on a small sample covering both numerical and non-numerical claims: 564 claims, sampled as in the human–judge agreement study (Table 8) across

Statistic	Estimate	95% CI
Rank-1 agreement	66.7%	[48.8, 80.8]
Rank-2 agreement	36.7%	[21.9, 54.5]
Rank-3 agreement	50.0%	[33.2, 66.8]
Exact full-ranking match	33.3%	[19.2, 51.2]
Mean Spearman ρ	0.483	[0.283, 0.667]
Raw agreement (vs. prompting)	90.0%	[74.4, 96.5]
Raw agreement (vs. BoT)	70.0%	[52.1, 83.3]
Win: AnalysisBank vs. prompting	91.7%	[83.3, 98.3]
Win: AnalysisBank vs. BoT	81.7%	[71.7, 91.7]

Table 10: Inter-annotator agreement and pairwise preference statistics with bootstrap 95% CIs (N=30). Chance level for the rank agreements is 33.3%.

	AnalysisBank		pooled baseline
Backbone	all claims	novel claims	all claims
Qwen3-8B	15.3% (23/150)	28.6% (8/28)	5.5% (7/127)
DeepSeek-V4-Flash	4.7% (7/150)	9.4% (3/32)	6.6% (9/137)

Table 11: Human factual audit of generated claims: contradiction rate (contradicted claims / audited claims), annotator fixed across backbones.

AnalysisBank and the baselines on Qwen3-8B and DeepSeek-V4-Flash, are each labelled *supported*, *contradicted* by the source, or *unverifiable* (blind to system identity; annotator fixed across backbones). We report the contradiction rate (Table 11); the baselines' novel claims are too few for a meaningful rate and are skipped. On the stronger backbone, AnalysisBank's contradiction rate is low (4.7% over all claims; 9.4% on novel claims) and below the pooled baseline (6.6%). On Qwen3-8B it is higher (15.3%; 28.6% on novel claims) while the baseline is stable (5.5%): valid inference tracks the reasoning capability of the backbone, most strongly for novel claims. Most novel claims are unverifiable against the source alone (61–69%), as they state content the input does not.